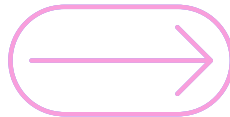


Programacion I COM - 03



INFORME DE PROYECTO

Soto Celeste - valensoto1305@gmail.com - 47095461

Speso Maylen - spesomay@gmail.com - 46564438

Profesores: Bidart Lucas y Davalos Leonardo

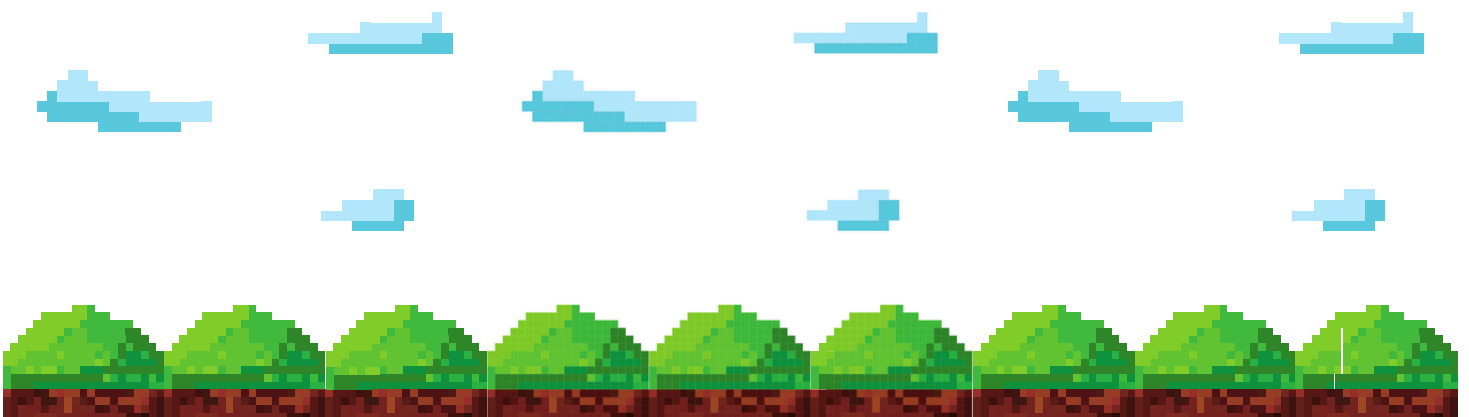


INTRODUCCIÓN

En este trabajo práctico se desarrolló un videojuego en Java utilizando Programación Orientada a Objetos. El juego se encuentra estructurado en distintas clases que representan los personajes, enemigos, plataformas y demás elementos del entorno.

Durante su desarrollo se aplicaron conceptos como clases, objetos, herencia, manejo de eventos de teclado y uso de imágenes para la creación de una interfaz gráfica. El objetivo fue integrar estos conceptos en un proyecto funcional e interactivo.

A continuación, se presenta el manual de usuario del videojuego. En esta sección se describe el funcionamiento general del juego, los controles disponibles para el jugador, los objetivos que debe cumplir para avanzar y las distintas mecánicas implementadas, como el movimiento de la princesa, los ataques, la aparición de enemigos, la obtención de vidas y la batalla contra el jefe final.





MANUAL DE USUARIO

Controles

El jugador puede controlar a la princesa mediante teclado y mouse:

- Flecha izquierda / A: mover hacia la izquierda
- Flecha derecha / D: mover hacia la derecha
- Flecha arriba / W: saltar
- Click izquierdo del mouse: disparar proyectiles
- Tecla 'R': reiniciar el juego instantáneamente desde la pantalla de victoria o derrota.

Mecánicas del juego

Movimiento:

- La princesa puede desplazarse por el escenario y saltar entre plataformas. El mapa se desplaza a medida que el jugador avanza.

Sistema de vida

El jugador comienza con una cantidad limitada de vidas.

- Cada choque con enemigos o daño recibido reduce una vida. Si las vidas llegan a cero, el juego finaliza.

Enemigos

Los enemigos aparecen a lo largo del mapa.

- Se mueven por el escenario.
- Si colisionan con la princesa, le quitan vida.
- Pueden ser eliminados con proyectiles.



ORGANIZACIÓN DEL PROYECTO

El programa está dividido en diferentes carpetas para organizar mejor los archivos y facilitar su mantenimiento.



PAQUETE ENTIDADES

- • CajaMagica.java
- • Castillo.java
- • Enemigo.java
- • EnemigoSaltarin
- • Entidad.java
- • Plataforma.java
- • Princesa.java
- • Proyectoil.java
- • ProyectoilEnemigo.java
- • Jefe.java

Contiene las clases que representan los distintos elementos del juego. Estas clases definen los personajes, objetos y elementos con los que interactúan el jugador

Contiene la clase encargada de la ejecución y control del juego. Esta clase gestiona la lógica principal, la interfaz gráfica y la interacción del usuario.



PAQUETE JUEGOS

- • Juego.java

Contiene una sola clase. Esta clase almacena valores y constantes utilizados en distintas partes del programa

Contiene las imágenes utilizadas por el juego, como personajes, enemigos, plataformas, fondo y otros elementos gráficos necesarios para la interfaz visual



PAQUETE UTIL

- • Constante.java



CARPETA IMG



DESCRIPCIÓN DE LAS CLASES

El programa está dividido en diferentes carpetas para organizar mejor los archivos y facilitar su mantenimiento.

PAQUETE ENTIDADES

- Entidad.java

La clase **Entidad** es la clase base del juego. Contiene los atributos comunes que utilizan los distintos objetos, como la posición, el tamaño y la velocidad de movimiento.

Métodos principales:

- **getBounds()**: devuelve el área ocupada por la entidad para detectar colisiones.
- **Métodos get y set**: permiten obtener y modificar la posición de la entidad.



DESCRIPCIÓN DE LAS CLASES



PAQUETE ENTIDADES



- Princesa.java

La clase **Princesa** representa al personaje principal controlado por el jugador.

Variables de instancia:

- **vidas**: cantidad de vidas disponibles.
- **saltando**: indica si la princesa está saltando.
- **cayendo**: indica si la princesa está cayendo.
- **imagen**: almacena la imagen del personaje.

Métodos principales:

- **moverIzquierda()**: desplaza a la princesa hacia la izquierda.
- **moverDerecha()**: desplaza a la princesa hacia la derecha.
- **detener()**: detiene el movimiento horizontal.
- **saltar()**: inicia el salto del personaje.
- **actualizar()**: actualiza la posición y el estado de la princesa.
- **perderVida()**: reduce una vida.
- **ganarVida()**: aumenta una vida.
- **getVidas()**: devuelve la cantidad de vidas actuales.
- **dibujar()**: muestra la princesa en pantalla.
- **getBoundsCuerpo()** y **getBoundsPies()**: se utilizan para detectar colisiones con mayor precisión.



DESCRIPCIÓN DE LAS CLASES

PAQUETE ENTIDADES

 • Enemigo.java

La clase **Enemigo** representa a los enemigos básicos del juego. Hereda de la clase Entidad, por lo que utiliza sus atributos de posición, tamaño y velocidad.

Variables de instancia:

- **vivo**: indica si el enemigo sigue activo en el juego.
- **imagen**: almacena la imagen que representa al enemigo.

Métodos principales:

- **actualizar()**: mueve al enemigo según su velocidad.
- **estaVivo()**: devuelve si el enemigo sigue vivo.
- **matar()**: cambia el estado del enemigo a eliminado.
- **getBoundsReal()**: devuelve el área ocupada por el enemigo para detectar colisiones.
- **dibujar()**: muestra el enemigo en pantalla utilizando su imagen.

Al crearse, el enemigo recibe una dirección de movimiento y se desplaza automáticamente por el escenario.



DESCRIPCIÓN DE LAS CLASES



PAQUETE ENTIDADES



- CajaMagica.java

La clase **CajaMagica** representa un objeto especial que puede ser utilizado por el jugador durante la partida. Hereda de la clase Entidad, por lo que posee una posición y dimensiones dentro del escenario.

Variables de instancia:

- **imagen**: almacena la imagen de la caja mágica.
- **usada**: indica si la caja ya fue utilizada.

Métodos principales:

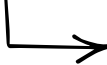
- **estaUsada()**: devuelve si la caja ya fue utilizada.
- **usar()**: cambia el estado de la caja a utilizada.
- **getBoundsReal()**: devuelve el área ocupada por la caja para detectar colisiones.
- **dibujar()**: muestra la caja en pantalla mientras no haya sido utilizada.

Cuando la caja es utilizada, deja de mostrarse en el juego.



DESCRIPCIÓN DE LAS CLASES

PAQUETE ENTIDADES

-  Castillo.java

La clase **Castillo** representa el objetivo final del juego.
Es el lugar al que debe llegar la princesa para completar el nivel.

Variables de instancia:

- ***imagen***: almacena la imagen del castillo.

El castillo es un elemento estático del escenario y funciona como la meta principal que debe alcanzar el jugador.

Métodos principales:

- ***getBoundsReal()***: devuelve el área ocupada por el castillo para detectar colisiones.
- ***dibujar()***: muestra el castillo en pantalla según la posición actual del mapa.



DESCRIPCIÓN DE LAS CLASES



PAQUETE ENTIDADES



- Plataforma.java

La clase **Plataforma** representa las superficies sobre las que puede desplazarse la princesa durante el juego. Hereda de la clase Entidad y utiliza sus atributos de posición y tamaño.

Variables de instancia:

- **imagen**: almacena la imagen utilizada para representar la plataforma.

Métodos principales:

- **getBoundsReal()**: devuelve el área ocupada por la plataforma para detectar colisiones.
- **dibujar()**: muestra la plataforma en pantalla utilizando la imagen correspondiente.

Las plataformas forman parte del escenario y permiten que el personaje pueda caminar, saltar y apoyarse sobre ellas.



DESCRIPCIÓN DE LAS CLASES

PAQUETE ENTIDADES

- Jefe.java

La clase **Jefe** representa al enemigo final del juego. Hereda de Entidad y agrega comportamiento especial, como mayor resistencia, movimiento flotante y ataques a distancia.

Variables de instancia principales

- **vidas**: cantidad de vidas del jefe.
- **imagen**: imagen utilizada para dibujar al jefe.
- **contadorAtaque**: controla la frecuencia de los ataques. **direccionY**: indica si el jefe sube o baja durante su movimiento.

Métodos principales

- **Jefe(int x, int y)**: inicializa la posición, tamaño, vidas y carga la imagen del jefe.
- **actualizar()**: mueve al jefe verticalmente y actualiza el contador de ataque.
- **intentarAtacar(int princesaX, int princesaY, int desplazamientoMapa)**: genera un **ProyectilEnemigo** dirigido hacia la princesa cada cierto tiempo.
- **perderVida()**: reduce en uno las vidas del jefe.
- **getVidas()**: devuelve la cantidad de vidas restantes.
- **getBoundsReal(int desplazamientoMapa)**: obtiene el área de colisión real del jefe en pantalla.
- **dibujar(Graphics g, int desplazamientoMapa)**: dibuja al jefe y su barra de vida.

Esta clase implementa al jefe final del juego, combinando movimiento, ataques, resistencia y representación gráfica para crear un desafío más importante que los enemigos comunes.



DESCRIPCIÓN DE LAS CLASES



PAQUETE ENTIDADES



- `ProyectilEnemigo.java`

La clase **ProyectilEnemigo** representa los proyectiles lanzados por los enemigos del juego, especialmente por el jefe final. Hereda de Entidad y se encarga de calcular la trayectoria y el movimiento del proyectil hacia la princesa.

Variables de instancia principales

- **dx**: componente horizontal de la dirección del proyectil.
- **dy**: componente vertical de la dirección del proyectil.

Métodos principales

- **ProyectilEnemigo(int origenX, int origenY, int destinoX, int destinoY)**: inicializa la posición del proyectil y calcula la dirección que debe seguir para dirigirse hacia la princesa.
- **actualizar()**: actualiza la posición del proyectil, desplazándolo en la dirección calculada.
- **dibujar(Graphics g)**: representa gráficamente el proyectil en pantalla mediante un círculo de color magenta.

Esta clase permite que los enemigos realicen ataques a distancia, agregando un mayor nivel de dificultad y dinamismo al juego.



DESCRIPCIÓN DE LAS CLASES



PAQUETE ENTIDADES



- `Proyectil.java`

La clase **Proyectil** representa los disparos que se generan durante el juego. Hereda de la clase Entidad y se mueve desde un punto de origen hacia un destino determinado.

Variables de instancia:

- **dx**: componente horizontal del movimiento.
- **dy**: componente vertical del movimiento.
- **activo**: indica si el proyectil sigue en uso.

Métodos principales:

- **actualizar()**: actualiza la posición del proyectil.
- **estaActivo()**: devuelve si el proyectil continúa activo.
- **destruir()**: desactiva el proyectil.
- **dibujar()**: representa el proyectil en pantalla mediante un círculo de color amarillo.

Al crearse, el proyectil calcula la dirección hacia la que debe desplazarse para llegar al destino indicado.

Resumen del paquete entidades

Las clases del paquete entidades representan todos los elementos que aparecen e interactúan dentro del juego. Cada una cumple una función específica, como controlar al personaje principal, gestionar enemigos, definir objetos interactivos o establecer la meta del nivel. Gracias a esta organización, el código resulta más claro y fácil de mantener.



DESCRIPCIÓN DE LAS CLASES



PAQUETE JUEGO



- juego.java

Contiene Juego.java, que concentra toda la lógica de control, el ciclo de ejecución (tick), el manejo de entradas y la gestión de la interfaz gráfica.

Funciones Principales:

- **Ciclo de Juego (Game Loop):** A través del método tick(), la clase actúa como un reloj interno que se actualiza constantemente. Es el encargado de llamar a los métodos de actualización de cada entidad, procesar las colisiones y repintar la pantalla para que el movimiento sea fluido.
- **Coordinación de Colisiones:** Actúa como el árbitro del juego mediante verificarColisiones(). Evalúa constantemente si la princesa interseca con enemigos, proyectiles o plataformas, gestionando la pérdida de vidas o la eliminación de entidades según sea necesario.
- **Lógica de Estados (Máquina de estados):** Controla el flujo de la partida mediante variables booleanas como ganado, perdido y batallaJefeIniciada. Esto permite que el juego transicione entre diferentes pantallas (juego activo, pantalla de derrota o pantalla de victoria) de forma automática.



DESCRIPCIÓN DE LAS CLASES



PAQUETE UTII



- Constantes.java

La clase **Constantes** almacena los valores fijos utilizados en distintas partes del juego. Su función es centralizar la configuración general para facilitar su mantenimiento y evitar la repetición de valores dentro del código.

Constantes principales:

- **ANCHO**: define el ancho de la ventana del juego.
- **ALTO**: define el alto de la ventana del juego.
- **FPS**: establece la cantidad de cuadros por segundo que ejecuta el juego.
- **GRAVEDAD**: representa la fuerza de gravedad aplicada a los personajes.
- **MAX_ENEMIGOS**: indica la cantidad máxima de enemigos que pueden existir simultáneamente.
- **LARGO_MAPA**: define la longitud total del escenario del juego.

Esta clase permite organizar y administrar los valores globales del proyecto, facilitando futuras modificaciones sin necesidad de cambiar múltiples partes del código

Resumen del paquete Util

El paquete util contiene las clases encargadas de almacenar valores y configuraciones utilizadas en distintas partes del juego. Estas clases permiten centralizar datos importantes como dimensiones de la ventana, gravedad, cantidad máxima de enemigos y tamaño del mapa, facilitando la organización del código y el mantenimiento del proyecto.



IMPLEMENTACIÓN

En esta sección se presentan algunos fragmentos del código fuente utilizados para desarrollar las funcionalidades principales del juego.

Movimiento de la princesa



```
public void manejarMovimiento() {
    princesa.detener();

    if (teclado.izquierda && princesa.getX() > 0) {
        princesa.moverIzquierda();
    }

    if (teclado.derecha) {
        if (princesa.getX() >= 500 && desplazamientoMapa < Constantes.LARGO_MAPA - Constantes.ANCHO) {
            desplazamientoMapa += 5;
        } else {
            princesa.moverDerecha();
        }
    }

    if (teclado.salto) {
        princesa.saltar();
    }
}
```

Este método controla el movimiento de la princesa mediante el teclado. Permite desplazarse hacia la izquierda y derecha, realizar saltos y generar el desplazamiento del mapa cuando el personaje avanza por el escenario.

Disparo con el mouse



```
if (mouse.click && proyectil == null) {
    proyectil = new Proyectil(
        princesa.getX() + 25,
        princesa.getY() + 25,
        mouse.x,
        mouse.y
    );
    mouse.click = false;
}
```

Este código permite crear un proyectil cuando el jugador realiza un clic con el mouse. La dirección del disparo se calcula a partir de la posición donde se hizo clic.

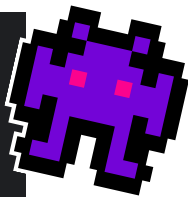


IMPLEMENTACIÓN

En esta sección se presentan algunos fragmentos del código fuente utilizados para desarrollar las funcionalidades principales del juego.

Colisión entre proyectil y enemigo

```
if (proyectil != null &&
    proyectil.getBounds().intersects(
        enemigos[i].getBoundsReal(desplazamientoMapa))) {
    enemigos[i] = null;
    proyectil = null;
}
```



Este fragmento verifica si un proyectil impacta contra un enemigo. Cuando ocurre la colisión, tanto el enemigo como el proyectil son eliminados.

Batalla contra el Jefe final



```
// Colisión de TU proyectil contra el JEFE
if (jefe != null && proyectil != null) {
    if (proyectil.getBounds().intersects(
        jefe.getBoundsReal(desplazamientoMapa))) {
        jefe.perderVida();
        proyectil = null; // Tu bala desaparece

        if (jefe.getVidas() <= 0) {
            jefe = null; // El jefe muere
        }
    }
}
```

Este código controla los impactos sobre el jefe final. Cada proyectil reduce sus vidas y, cuando estas llegan a cero, el jefe es derrotado.



PROBLEMAS, DECISIONES Y SOLUCIONES DURANTE EL DESARROLLO

Problemas:

Durante el desarrollo del proyecto surgieron algunos inconvenientes relacionados principalmente con el trabajo colaborativo mediante Git y GitHub.

Problema 1:

- Conflictos al subir cambios al repositorio. En varias ocasiones, al subir modificaciones al repositorio, algunos commits anteriores parecían desaparecer o eran reemplazados por versiones más recientes del proyecto.

Solución:

- Se revisó la forma de utilizar Git, realizando actualizaciones del repositorio antes de subir cambios y verificando el historial de commits para evitar sobrescribir el trabajo realizado por otros integrantes.

Problema 2:

- Aparición de conflictos de fusión (emerge conflictos). Al descargar o actualizar el proyecto aparecían líneas como <<<<<< HEAD, ===== y >>>>>>, generadas cuando Git detectaba cambios diferentes en un mismo archivo.

Solución:

- Se identificaron las secciones en conflicto y se realizó una revisión manual del código para conservar las modificaciones correctas y eliminar las marcas generadas por Git.



PROBLEMAS, DECISIONES Y SOLUCIONES DURANTE EL DESAROLLO

Problema 3:

- Uno de los mayores desafíos fue lograr que la princesa se moviera correctamente por el escenario. Al principio, en algunas situaciones atravesaba las plataformas o no detectaba bien cuándo estaba apoyada sobre ellas.

Solución:

- Se ajustó el sistema de colisiones y la gravedad del personaje, creando zonas específicas para detectar el cuerpo y los pies de la princesa. De esta manera, se consiguió un movimiento más preciso y natural durante el juego.



CONCLUSIÓN

El desarrollo de este proyecto nos permitió aplicar los conocimientos de Programación Orientada a Objetos para crear un videojuego funcional e interactivo. A través de la implementación de mecánicas como el movimiento del personaje, enemigos con física propia, proyectiles y un jefe final con ataques a distancia, logramos integrar diversos conceptos técnicos en una arquitectura coherente. Asimismo, la resolución de desafíos como la estabilidad visual del fondo y el ajuste preciso de colisiones nos brindó experiencia práctica en la optimización del código y la gestión de recursos gráficos. Finalmente, el trabajo colaborativo y la resolución de conflictos en el repositorio fortalecieron nuestras capacidades para enfrentar problemas técnicos en equipo, culminando en un producto estable y con una experiencia de usuario optimizada mediante funciones de reinicio rápido.

